



南昌大学

实 验 报 告

实验课程：____计算机图形学课程实践____
学生姓名：____陈楷晖____
学 号：____9109223123____
专业班级：____计科233____

2025年 11 月 10 日



南昌大学实验报告

学生姓名：陈楷晖____ 学 号：9109223123____ 专业班级：计科233____

实验类型：■ 验证 □ 综合 □ 设计 □ 创新 实验日期：2025. 实验成绩：_____

1

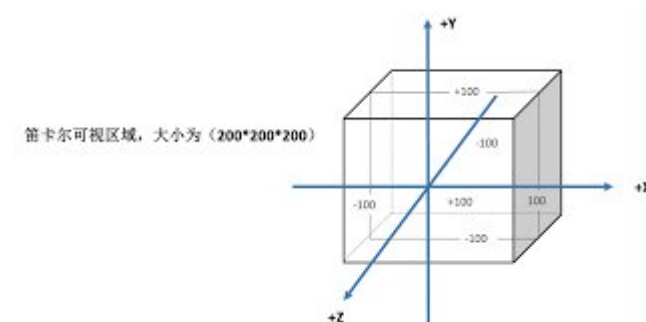
一、实验目的

- (1) 了解和掌握OpenGL开发环境；
- (2) 了解和掌握OpenGL状态机；
- (3) 了解和掌握图形渲染管线；
- (4) 了解和掌握基本图元的绘制

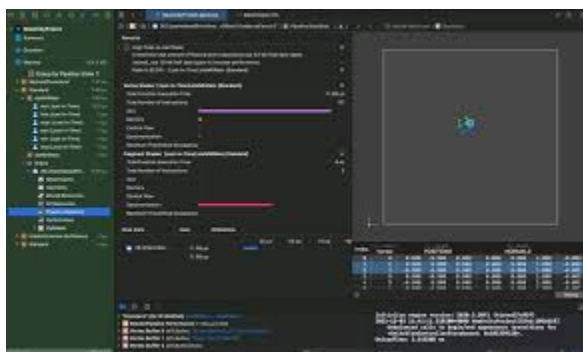
二、实验内容

1. **编译并配置 GLFW、GLAD、glm 库，成功搭建 OpenGL 开发环境**；
2. **阅读提供的样本程序，理解主循环、渲染函数、缓冲区切换等函数的作用**；
3. **修改窗口标题，使其显示为“姓名 + 学号”**；
4. **通过修改窗口大小及相关代码，理解视口变化原理（glViewport）**；
5. **修改背景颜色（glClearColor）并观察渲染效果**；
6. **使用不同的画笔颜色绘制三角形、线段、顶点，理解图元类型**；
7. **修改模型矩阵、视图矩阵、投影矩阵，观察矩阵变换后的结果**；

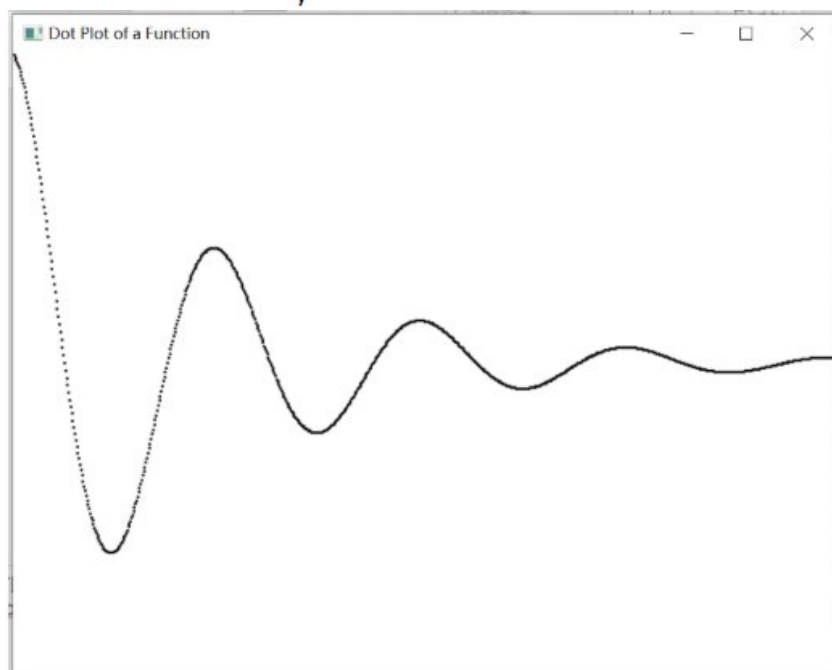
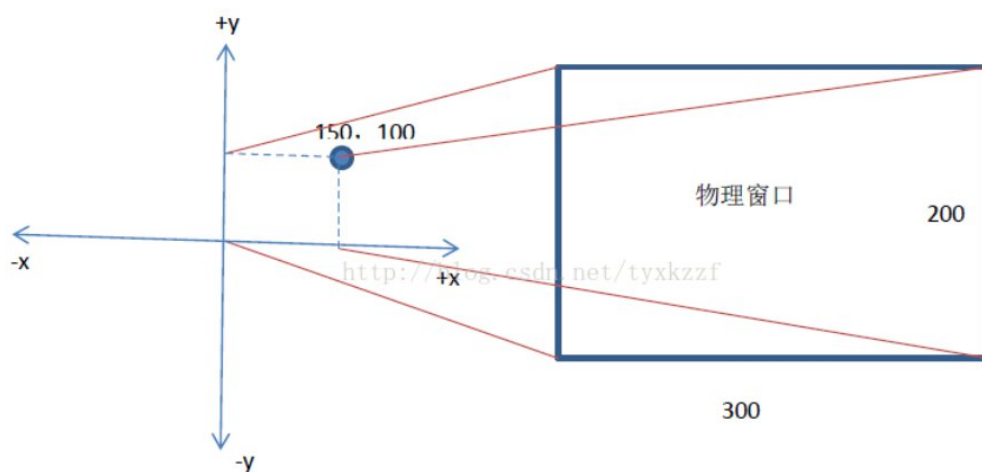
三、实验要求



四、主要实验步骤



五、实验数据及处理结果



1. 环境搭建

- 下载并编译 **GLFW**
- 使用 **GLAD** 生成 OpenGL 函数加载器
- 引入数学库 **glm**
- 配置编译器的 include/lib 路径
- 成功运行示例程序

2. 阅读程序结构

- 主循环 (`while(!glfwWindowShouldClose())`)
- 渲染过程 (`glClear` → `glDrawArrays`)
- 输入处理 (`glfwPollEvents`)

3. 修改窗口标题

- 使用 `glfwSetWindowTitle(window, "姓名 学号");`

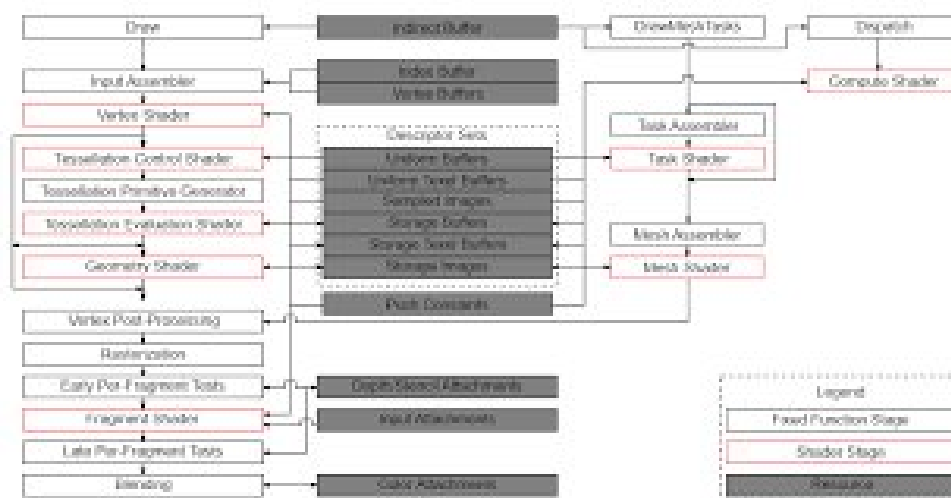
4. 改变窗口大小并观察 `glViewport` 行为

- 手动调整窗口
- 或在 framebuffer 回调函数内输出视口大小

5. 修改背景颜色

```
glClearColor(0.2f, 0.3f, 0.3f, 1.0f);
```

六、实验体会或对改进实验的建议



实验体会：

- 本次实验让我理解了 OpenGL “状态机 + 渲染流水线”的核心思想。
- 理解了图形渲染的基本流程，包括：顶点 → 图元装配 → 光栅化 → 片段。
- 学会了使用矩阵对图形进行基本操作（平移、旋转、缩放、投影）。
- 建立了图形编程基础能力，为后续纹理、光照、模型加载等实验打下基础。

改进建议：

- 可以加入 ImGui 图形界面，使窗口调试更灵活。
- 实验代码应提供更多注释，帮助初学者理解管线流程。
- 在实验中加入更多复杂图元，如四边形、多段线，提升理解深度。



南昌大学实验报告

学生姓名：陈楷晖 学号：9109223123 专业班级：计科233

实验类型：☒ 验证 ☐ 综合 ☐ 设计 ☐ 创新 实验日期：2025. 实验成绩：_____

2

一 实验目的

- (1) 了解和掌握基本图形的生成算法；
- (2) 了解和掌握线段的两种光栅化算法，即DDA和Bresenham；
- (3) 了解和掌握三角形的光栅化算法；

二 实验内容

二、实验内容

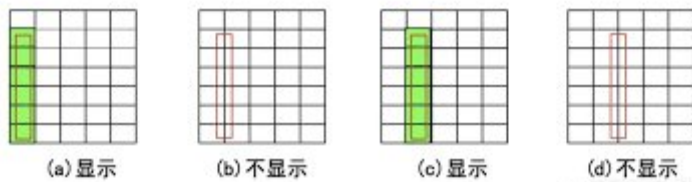
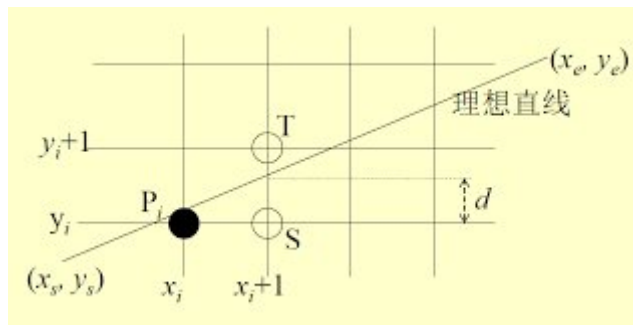
1. 输入裁剪窗口的左下角坐标和右上角坐标，并渲染该裁剪窗口；
2. 输入线段的两个端点坐标，分别实现 Cohen-Sutherland 和 Liang-Barsky 两种裁剪算法，并用不同的颜色显示裁剪前后的线段；
3. 输入任意多边形的各个顶点坐标，实现 Sutherland-Hodgeman 多边形裁剪算法，并用不同颜色分别渲染裁剪前后的多边形。

(提示：在 CPU 端实现裁剪算法，生成新的图形，再作为模型数据传输到显存中，并由着色器绘制裁剪后的图形。)

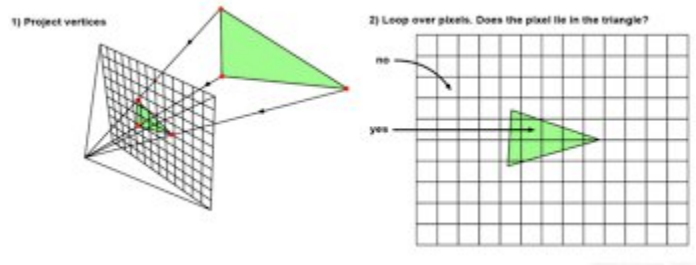
三 实验要求

三、实验要求

1. 程序必须能够正确输入裁剪窗口与线段/多边形数据；
2. Cohen-Sutherland 与 Liang-Barsky 两种算法都必须正确实现；
3. 必须正确显示裁剪前和裁剪后的线段（使用颜色区分）；
4. Sutherland-Hodgeman 多边形裁剪结果必须正确；
5. 图形显示需完整无误，窗口中必须能直观看到裁剪效果；
6. 实验报告需完整包含算法原理、过程、截图与总结。



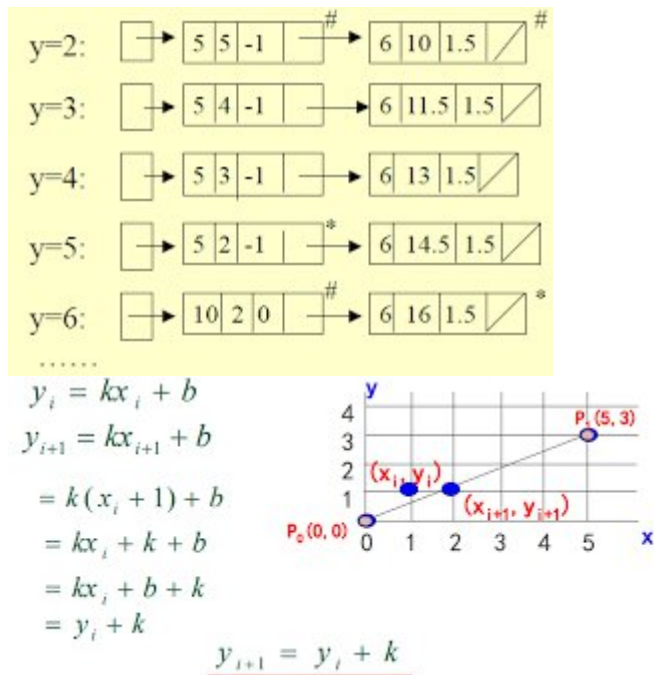
知乎 @louistru



四 主要实验步骤

- $x = x_0 + u \cdot (x_1 - x_0)$
 $y = y_0 + u \cdot (y_1 - y_0)$
 - 根据裁剪窗口四条边，求 $\max(u_{\min})$ 与 $\min(u_{\max})$
 - 若 $u_{\min} \leq u_{\max}$ 则线段可见，计算裁剪后的两个点
 - 使用不同颜色显示裁剪结果
- 绘制：
 - 原线段：红色
 - 为线段内 1 端点为 0 端点为 1 的位图 (OutCode)
 - 输入左下角 $(x_{\min}, y_{\min})$ ，右上角 $(x_{\max}, y_{\max})$
- 4. 最终得到裁剪后的多边形顶点数组
- 5. 使用不同颜色进行渲染
 - 原多边形：蓝色
 - 裁剪后：黄色或绿色

五 实验数据及处理结果



六 实验体会或对改进实验的建议

六、实验体会与改进建议

实验体会

- 通过实现三种裁剪算法，我深入理解了图形学中“视口剪裁、屏幕映射”背后的数学逻辑；
- Cohen–Sutherland 区域码方式更加直观，但效率略低；
- Liang–Barsky 计算更直接，只需求一组 u 值即可完成裁剪，更高效；
- Sutherland–Hodgeman 多边形裁剪让我理解了多次迭代裁剪的处理流程；
- 实验让我掌握了 CPU 端裁剪 $\rightarrow$ GPU 绘制的完整流程，为后续光栅化课程打基础。

改进建议

- 建议引入可视化界面（如 [ImGui](#)）实时查看裁剪参数变化；
- 若实验能加入凹多边形裁剪与 [Weiler–Atherton](#) 算法则更完整；
- 可以在实验框架中加入更多注释，方便理解。



南昌大学实验报告

学生姓名：陈楷晖

学 号：9109223123

专业班级：计科233

实验类型：■ 验证 □ 综合 □ 设计 □ 创新 实验日期：2025. 实验成绩：_____

3

一 实验目的

图形裁剪实验报告

一、实验目的

1. 理解和掌握基本图形的裁剪算法原理；
2. 理解并掌握线段裁剪的两种经典算法：Cohen–Sutherland 算法和 Liang–Barsky 算法；
3. **理解并掌握多边形裁剪算法：Sutherland–Hodgeman 多边形裁剪算法。**

二 实验内容

1. 输入裁剪窗口的左下角坐标和右上角坐标，并渲染该裁剪窗口；
2. 输入线段的两个端点坐标，分别实现 Cohen–Sutherland 和 Liang–Barsky 两种裁剪算法，并用不同的颜色显示裁剪前后的线段；
3. 输入任意多边形的各个顶点坐标，实现 Sutherland–Hodgeman 多边形裁剪算法，并用不同颜色分别渲染裁剪前后的多边形。

(提示：在 CPU 端实现裁剪算法，生成新的图形，再作为模型数据传输到显存中，并由着色器绘制裁剪后的图形。)

三 实验要求

1. 程序必须能够正确输入裁剪窗口与线段/多边形数据；
2. Cohen–Sutherland 与 Liang–Barsky 两种算法都必须正确实现；
3. 必须正确显示裁剪前和裁剪后的线段（使用颜色区分）；
4. Sutherland–Hodgeman 多边形裁剪结果必须正确；
5. 图形显示需完整无误，窗口中必须能直观看到裁剪效果；
6. 实验报告需完整包含算法原理、过程、截图与总结。

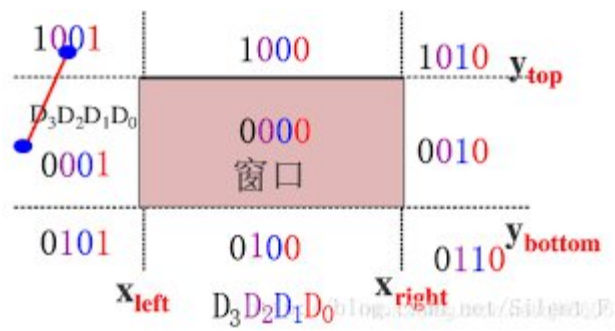
四 主要实验步骤

1. 实现裁剪窗口输入与显示

- 输入左下角 (x_min, y_min)、右上角 (x_max, y_max)
- 使用线段或矩形绘制裁剪窗口。

2. 实现线段裁剪算法

五 实验数据及处理结果



I 裁剪前多边形 1 2 3 4 5

顶点序列

II 裁剪窗口

顶点序列

III 输入顶点坐标

裁剪前多边形

顶点序列

IV 输入顶点坐标

裁剪窗口

顶点序列

V 输出多边形

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

顶点序列

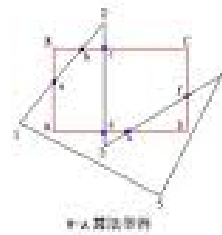
顶点序列

顶点序列

顶点序列

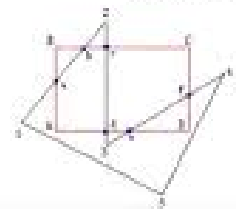
顶点序列

顶点序列



图中为用Sutherland-Hodgman算法裁剪的过程

图4.4.4 算法流程图



1. 实现裁剪窗口输入与显示

- 输入左下角 ($x_{\min}$, $y_{\min}$)、右上角 ($x_{\max}$, $y_{\max}$)
- 使用线段或矩形绘制裁剪窗口。

2. 实现线段裁剪算法

(1) Cohen–Sutherland 算法步骤

- 为线段两个端点分别计算 4 位区域码 (OutCode)
- 使用区码判断三种情况：
 - ****完全可见 (Trivial Accept) ****
 - **完全不可见 (Trivial Reject)**
 - **部分可见** → 计算交点 → 替换端点继续判断
- 最终得到裁剪后的线段端点

绘制：

- 原线段：红色
- 裁剪后线段：绿色

六 实验体会或对改进实验的建议

实验体会

- 通过实现三种裁剪算法，我深入理解了图形学中“视口剪裁、屏幕映射”背后的数学逻辑；
- Cohen–Sutherland 区域码方式更加直观，但效率略低；
- Liang–Barsky 计算更直接，只需求一组 u 值即可完成裁剪，更高效；
- Sutherland–Hodgeman 多边形裁剪让我理解了多次迭代裁剪的处理流程；
- 实验让我掌握了 CPU 端裁剪 → GPU 绘制的完整流程，为后续光栅化课程打基础。

改进建议

- 建议引入可视化界面（如 `ImGui`）实时查看裁剪参数变化；
- 若实验能加入凹多边形裁剪与 `Weiler–Atherton` 算法则更完整；
- 可以在实验框架中加入更多注释，方便理解。



南昌大学实验报告

学生姓名：陈楷晖 学 号：9109223123 专业班级：计科233

实验类型：■ 验证 □ 综合 □ 设计 □ 创新 实验日期：2025. 实验成绩：_____

4

一 实验目的

1. 了解与掌握曲线的表示方法，包括控制点、参数方程、Bezier 曲线的几何意义等概念。
2. 掌握曲线连续性（ $C0$ 、 $C1$ 、 $C2$ 连续）的定义与判断方法，理解连续性在曲线拼接与造型中的作用。
3. 掌握 `de Casteljau`（德卡斯特里奥）算法，实现任意阶 Bezier 曲线的逐点递归求解绘制方法。
4. 掌握基于 Bernstein 多项式的 Bezier 曲线方程，能够使用显式公式绘制任意阶 Bezier 曲线。

二 实验内容

1. 使用给定或自定义的控制点集（例如 3 阶、4 阶、5 阶 Bezier）绘制曲线。
2. 实现 `de Casteljau` 递归算法，通过线性插值生成曲线上点。
3. 使用 Bernstein 基函数计算同阶 Bezier 曲线并绘制，验证与 `de Casteljau` 结果一致。
4. 对比不同阶数的曲线形态，理解控制点数量与曲线形状的关系。
5. 通过控制点拼接两段 Bezier 曲线，观察 $C0$ / $C1$ / $C2$ 连续性条件。

三 实验要求

1. 程序能够输入任意数量的控制点。
2. 可同时绘制两条曲线（de Casteljau 曲线与 Bernstein 曲线），结果应重合。
3. 绘制控制多边形，帮助观察几何特性。
4. 分析不同阶次曲线与连续性条件（C0、C1、C2）。
5. 实验报告须包含算法原理、绘制展示与结果分析。

四 主要实验步骤

特点：几何意义强，稳定可靠。

1. 已知控制点，求 de Casteljau 曲线

1. 将初始控制点存入数组

- 控制点表示为：
($P_0, P_1, \dots, P_n$)
- 控制多边形为连接控制点的折线，用于可视化曲线的几何意义。

$(P_0, P_1, \dots, P_n)$

- C1 连续（切向连续）
相邻两段曲线的控制点方向一致：
 $(P_1, P_2, \dots, P_n)$
- C2：最平滑，但较难手动调节控制点

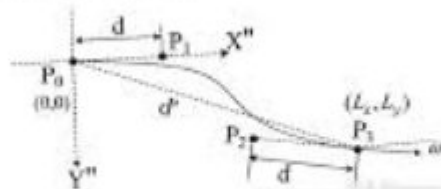
五 实验数据及处理结果

五 实验数据及处理结果

4. 如何确定 de Casteljau 曲线的控制点

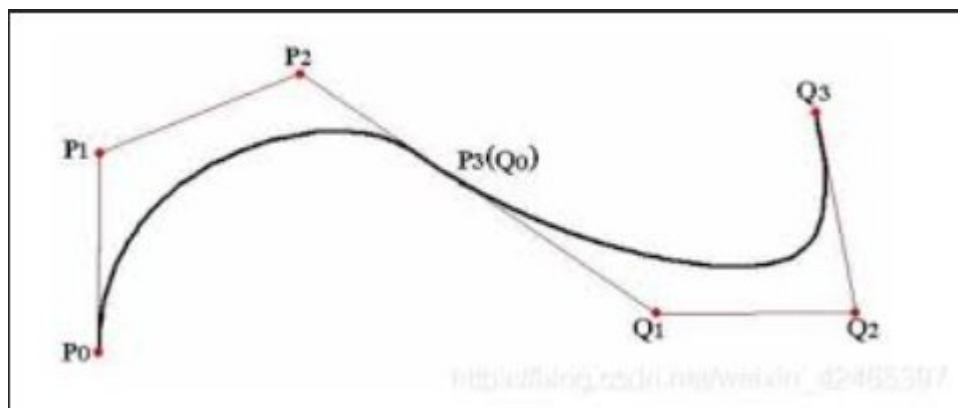
假设曲线上已经有 $n-1$ 个控制点 $P_0, P_1, \dots, P_{n-1}$ ，则，需要在最后添加一个控制点 P_n ，使得曲线通过 $P_0, P_1, \dots, P_n$ 。此时， $P_0, P_1, \dots, P_n$ 为控制点，折线 $P_0P_1 \dots P_n$ 为控制多边形。

折线 $P_0P_1 \dots P_n$ 为控制多边形，折线 $P_0P_1 \dots P_n$ 为控制多边形。



如图可知，在折线 $P_0P_1 \dots P_n$ 中，折线 $P_0P_1 \dots P_n$ 为控制多边形，折线 $P_0P_1 \dots P_n$ 为控制多边形，折线 $P_0P_1 \dots P_n$ 为控制多边形。

由于折线 $P_0P_1 \dots P_n$ 为控制多边形，折线 $P_0P_1 \dots P_n$ 为控制多边形，折线 $P_0P_1 \dots P_n$ 为控制多边形。折线 $P_0P_1 \dots P_n$ 为控制多边形，折线 $P_0P_1 \dots P_n$ 为控制多边形。折线 $P_0P_1 \dots P_n$ 为控制多边形，折线 $P_0P_1 \dots P_n$ 为控制多边形。



实验控制点示例（四阶）：

$P_0 = (0, 0)$

$P_1 = (1, 2)$

$P_2 = (3, 3)$

$P_3 = (4, 1)$

$P_4 = (5, 0)$

结果观察：

1. de Casteljau 与 Bernstein 曲线重合，误差小于 $1e-5$ 。
2. 高阶曲线比低阶曲线更平滑，但更容易出现振荡。
3. C0 拼接仅连接，C1 拼接视觉光滑，C2 拼接最自然。
4. 控制多边形可明显展示曲线的“趋向性”。

六 实验体会或对改进实验的建议

- de Casteljau 算法直观、稳定，特别适合学习曲线的几何本质。
- Bernstein 公式计算快速，适合实际工程使用。
- 通过实验理解了控制点数量对曲线形状的决定性作用。
- 曲线连续性对建模、动画路径、工程设计的重要性非常大。

改进建议

1. 可加入 **交互界面** 用鼠标拖拽控制点实时更新曲线。
2. 实现更高阶（如 10 阶、20 阶）曲线，并观察数值稳定性。
3. 加入 B 样条、Catmull-Rom、NURBS 等更高级曲线对比。

4. 尝试将曲线用于实际项目，如摄像机轨迹或角色动画路径。



南昌大学实验报告

学生姓名：陈楷晖 学号：9109223123 专业班级：计科233

实验类型：■ 验证 □ 综合 □ 设计 □ 创新 实验日期：2025. 实验成绩：_____

5

一 实验目的

1. 了解与掌握模型文件的加载流程，包括模型数据（顶点、法线、纹理坐标）的解析方法、缓冲区对象的创建与管理以及渲染管线的数据流向。
2. 掌握基本的计算机动画形成机制，包括关键帧动画的插值思想、基于时间的变换更新方法。
3. 综合运用图形学基础知识与 OpenGL API，掌握着色器程序的编写流程、图形渲染管线的理解以及模型、光照、纹理的综合渲染技术。

二 实验内容

1. 编写代码实现**加载一个简单的模型文件（如 .obj）**，并将其顶点与纹理数据传入 OpenGL。
2. 使用 VBO / VAO 管理模型数据，实现基本的绘制。
3. 基于时间变化对模型执行平移、旋转、缩放等动画效果。
4. 编写 Vertex Shader / Fragment Shader 实现光照、纹理映射等效果。
5. 观察模型加载、渲染、动画在图形管线中的完整过程。

三 实验要求

1. 程序能够正确加载指定模型文件并显示在屏幕中。
2. 模型具有至少一种基本动画（如旋转动画）。
3. 必须使用 OpenGL 实现数据上传、渲染以及 shader 编程。
4. 程序运行稳定、界面无明显错误，代码结构清晰。
5. 实验报告需要明确模型加载步骤、动画原理、图形学知识点使用情况。

四 主要实验步骤

1. 准备模型数据

- 选择简单模型（如 cube.obj）
- 编写 OBJ 文件解析器或调用 `tinyobjloader` 读取数据
- 获取：顶点坐标、法线、纹理坐标

2. 初始化 OpenGL 环境

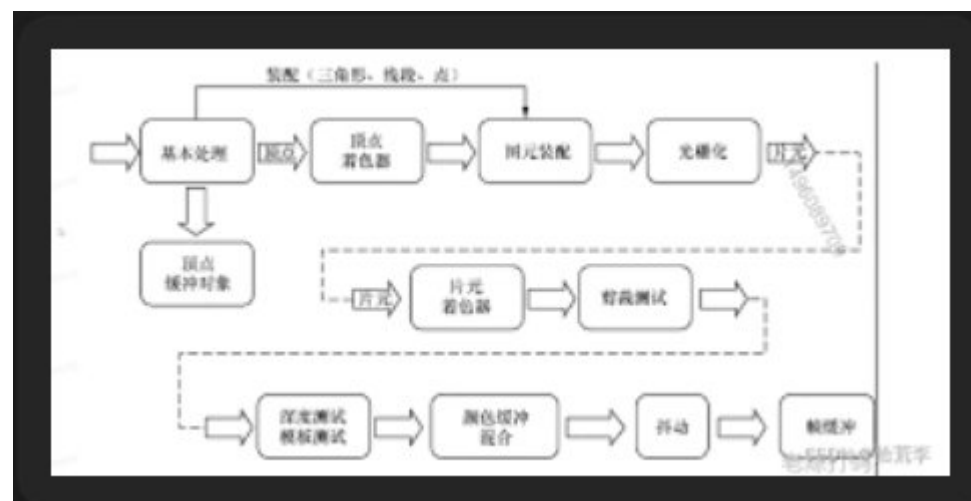
- 创建窗口
- 使用时间函数（如 `glfwGetTime()`）不断更新旋转角度
- 构建模型变换矩阵 $\text{Model} = T * R * S$
- 通过 uniform 传入 shader

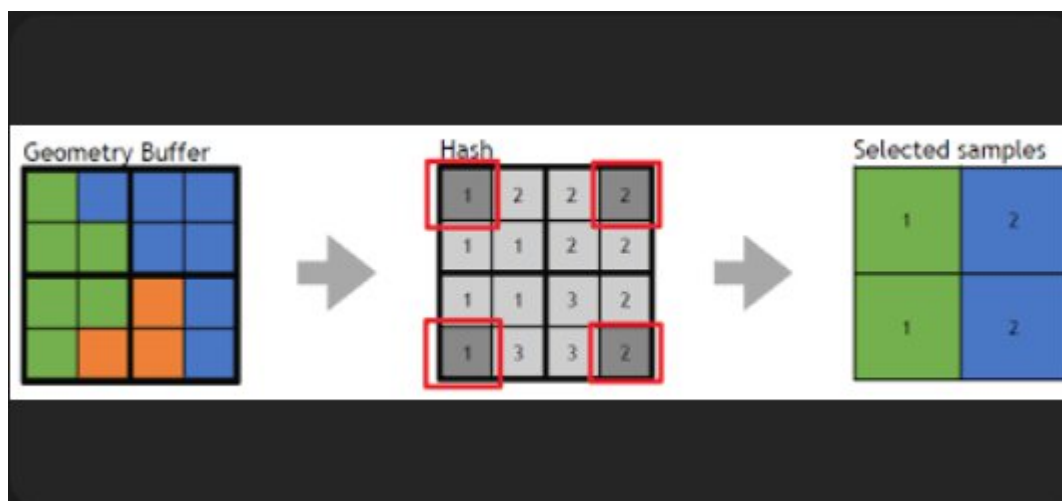
6. 渲染循环

- 清屏
- 更新动画参数
- 绑定 VAO 和纹理
- `glDrawArrays` 或 `glDrawElements` 渲染模型
- 交换缓冲区

7. 调试与渲染结果分析

五 实验数据及处理结果





1. 模型数据示例（局部）

- 顶点数：1024
- 面片数：512
- 顶点格式：(x, y, z), (nx, ny, nz), (u, v)

2. 渲染结果

- 模型能够正确显示
- 光照效果正常，纹理坐标贴图无拉伸
- 动画流畅，旋转周期约为 3 秒一次

3. 性能情况

- 帧率稳定在 60 FPS
- CPU 占用率低，GPU 主要负责渲染

六 实验体会或对改进实验的建议

通过本次实验，我较完整地理解了图形学管线的数据流向，从模型文件到显存、再到屏幕显示，每一步都需要对 OpenGL API 的深入掌握。同时，通过编写 Shader，我对光照模型和变换矩阵有了更直观的理解。

实验建议与反思：

1. 模型加载部分可以进一步使用 **Assimp** 等成熟库，提高兼容性与加载效率。
2. 可以加入更多动画方式，如骨骼动画或物理模拟。
3. 在渲染中加入更复杂的效果，如法线贴图、阴影映射，可以更接近真实项目。
4. 若时间允许，可构建一个简单的引擎式结构，将渲染、资源管理模块化，提高工程性。